

EC2 Dev Box Runbook

Standing up a remote Linux development machine on AWS, wiring it into VS Code, and serving something to the public internet — with every failure that actually happened and the error text it produced.

Built **2026-08-31**

Windows 10 · Git Bash + PowerShell

Ubuntu 24.04 · t3.micro

Region **us-east-1**

§ Fill these in first

Run this once per PowerShell session — everything below references these variables.

```
POWERSHELL your laptop

$ACCT = "123456789012"      # aws sts get-caller-identity --query Account --output
text
$KEY  = "my-laptop"        # name for your key pair in AWS
$IID  = "i-0123456789abcdef0" # instance id, once you have one
$SG   = "sg-0123456789abcdef0" # security group id
$VOL  = "vol-0123456789abcdef0" # root volume id
```

Find them any time:

```
POWERSHELL your laptop

aws sts get-caller-identity

aws ec2 describe-instances `
  --query 'Reservations[].Instances[.
{ID:InstanceId,State:State.Name,IP:PublicIpAddress,SG:SecurityGroups[0].GroupId}' `
  --output table
```

PLATFORM NOTE

Use PowerShell for `aws` commands. `aws.exe` is a Windows binary and won't understand Git Bash paths like `/c/Users/...`. It installs to `C:\Program Files\Amazon\AWSCLIV2\aws.exe`.

00 What EC2 is, and when it's worth it

EC2 is **a computer you rent by the second in a data center**. That's the whole idea. You SSH in and it's an ordinary Linux box — except it has a public IP and keeps running when your laptop is closed.

Use it when you need one of these:

- **It must stay up when your laptop sleeps** — web server, API, bot, cron job
- **It must be publicly reachable** — webhooks, a demo URL others can open
- **It must be bigger than your laptop** — lots of RAM, or a GPU, for a few hours
- **It must be disposable** — break it, terminate it, relaunch clean in 60 seconds
- **It must be a consistent Linux env** — the same box for the whole team

Otherwise, run it locally. It's just someone else's computer.

REALITY CHECK

A t3.micro has 1 GB RAM and 2 vCPU — *less capable than free Google Colab*. It's an excellent web server and Linux practice box; it is not an ML machine. For that you want a GPU instance (`g4dn.xlarge` , ~\$0.53/hr), which requires a quota-increase request before you can launch one.

§ Before you start — account, CLI, credentials

Everything below assumes `aws` already works on your laptop. Run this check first:

```
POWERSHELL your laptop
aws sts get-caller-identity
```

An account number and a user ARN means you're ready — skip to section 1. `command not found` or `Unable to locate credentials` means you need the route that matches your class.

Route A — your own AWS account

1. **Sign up** at aws.amazon.com. You need a card for identity verification, and AWS may place a temporary ~\$1 hold on it. Staying inside the free tier and following sections 10 and 12 keeps the actual bill at \$0.
2. **Stop using root immediately.** The account you just created is the root user and cannot be restricted. Go to **IAM** → **Users** → **Create user**, make yourself a normal user, and attach the `AmazonEC2FullAccess` policy — that covers sections 1 through 9. Turn on MFA while you're there.
3. **Create an access key.** IAM → your user → **Security credentials** → **Create access key** → **Command line interface (CLI)**. Copy both values now — the secret is shown exactly once and cannot be retrieved later.
4. Continue with **Both routes** below.

Route B — an IAM user in your instructor's account

Your instructor creates one IAM user per student and sends you an access key id and secret. You skip signup entirely — and your instructor sees and pays for every resource you launch, so stop your instance when you're done (section 12).

INSTRUCTOR SIDE

IAM → Users → Create user, one per student → attach `AmazonEC2FullAccess` → Security credentials → Create access key. Set a budget alarm (section 10) before handing any of them out.

Both routes — install the CLI and log in

Install **AWS CLI v2** from the Windows MSI at awscli.amazonaws.com/AWSCLIV2.msi, then **open a new terminal** so it lands on your PATH:

```
● POWERSHELL your laptop

aws --version           # aws-cli/2.x.x ...

aws configure
# AWS Access Key ID      -> paste
# AWS Secret Access Key -> paste
# Default region name    -> us-east-1
# Default output format -> json

aws sts get-caller-identity # confirm it worked
```

NEVER COMMIT OR SHARE AN ACCESS KEY

It is a password to your whole account. Keep it out of git, out of screenshots, out of shared drives. If one leaks, delete it in IAM immediately — and delete your keys at the end of term either way.

01 Generate an SSH key pair

● GIT BASH

your laptop

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/ec2-key -C "my-laptop" -N ""
```

In PowerShell the quoting differs — `-N ""` is *not* empty there:

● POWERSHELL

your laptop

```
New-Item -ItemType Directory -Force "$env:USERPROFILE\.ssh" | Out-Null
ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\ec2-key" -C "my-laptop" -N ''
```

- `ec2-key` — **private key, never share it**
- `ec2-key.pub` — public key, safe to paste anywhere

Omit `-N` entirely to be prompted for a passphrase — more secure, and you type it on each connect.

Fix the key permissions — Windows-specific, and it will bite you

SSH refuses to use a private key other accounts can read. On Windows, `chmod 600` from Git Bash often does **not** stick — you must set the NTFS ACL.

● GIT BASH

your laptop

```
export MSYS_NO_PATHCONV=1      # stop Git Bash rewriting /flags into paths
KEYPATH="C:\Users\%USERNAME%\\.ssh\ec2-key"
WINUSER=$(/c/Windows/System32/whoami.exe | tr -d '\r') # -> machine\user
icaccls "$KEYPATH" /reset
icaccls "$KEYPATH" /inheritance:r /grant:r "${WINUSER}:R"
icaccls "$KEYPATH"             # verify: exactly ONE entry, your account
```

THE TRAP

Use the full `machine\user`, never the bare username. Git Bash's `whoami` and PowerShell's `$env:USERNAME` both return only half of it. `icacls` accepts it, resolves it to a **bogus principal**, and combined with `/inheritance:r` locks *you* out of your own key:

```
Load key "C:\Users\...\ec2-key": Permission denied
ubuntu@1.2.3.4: Permission denied (publickey).
```

Recover by re-running `/reset` then `/grant:r` with the full `machine\user`.

02 Launch an instance

Register your public key with AWS

POWERSHELL

your laptop

```
aws ec2 import-key-pair --key-name $KEY `
  --public-key-material "fileb://$env:USERPROFILE/.ssh/ec2-key.pub"
```

`fileb://` (binary) rather than `file://` — the text form can mangle the key on Windows.

Look up the AMI — never hardcode it

AMI IDs are **region-specific and change constantly**. This box was built on `ami-0f8a61b66d1accaee`; weeks later the current Ubuntu 24.04 image was `ami-0d7f022123f8ff19d`. Always resolve it from an SSM public parameter.

POWERSHELL

your laptop

```
# Ubuntu 24.04 LTS
$AMI = aws ssm get-parameters `
  --names "/aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-
gp3/ami-id" `
  --query "Parameters[0].Value" --output text

# Amazon Linux 2023 alternative:
# --names "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"

echo $AMI
```

GIT BASH

Set `export MSYS_NO_PATHCONV=1` first, or the leading `/aws/...` becomes `C:/Program Files/Git/aws/...` and the call returns `InvalidParameters` with no error message.

Create a security group

POWERSHELL

your laptop

```
$myip = (Invoke-RestMethod https://checkip.amazonaws.com).Trim()

$SG = aws ec2 create-security-group --group-name dev-box-sg `
    --description "SSH from my IP, HTTP/HTTPS public" `
    --query "GroupId" --output text

# SSH: your IP only
aws ec2 authorize-security-group-ingress --group-id $SG `
    --protocol tcp --port 22 --cidr "$myip/32"

# HTTP/HTTPS: public (skip if you aren't serving anything)
aws ec2 authorize-security-group-ingress --group-id $SG --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $SG --protocol tcp --port 443 --cidr 0.0.0.0/0
```

Launch it

POWERSHELL

your laptop

```
$IID = aws ec2 run-instances `
    --image-id $AMI `
    --instance-type t3.micro `
    --key-name $KEY `
    --security-group-ids $SG `
    --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":20,"VolumeType":"gp3"}}]' `
    --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=dev-box}]" `
    --query "Instances[0].InstanceId" --output text

aws ec2 wait instance-status-ok --instance-ids $IID

aws ec2 describe-instances --instance-ids $IID `
    --query "Reservations[0].Instances[0].PublicIpAddress" --output text
```

TWO THINGS WORTH KNOWING

Use 20 GB, not the 8 GB default. An 8 GB root volume hits 95% full from just the OS, VS Code's remote server, and JupyterLab. The extra 12 GB costs about \$1/month.

`wait instance-status-ok` **blocks until the OS has actually booted.** `running` alone means the VM exists, not that sshd is listening.

03 Rescue an instance that has no key pair

If you launched from the console without selecting a key pair, you normally can't log in at all. **EC2 Instance Connect** rescues it by injecting a temporary key — valid **60 seconds** — authorised by your AWS credentials. Ubuntu and Amazon Linux AMIs ship with the agent.

POWERSHELL

your laptop

```
aws ec2 start-instances --instance-ids $IID
aws ec2 wait instance-status-ok --instance-ids $IID

$ip = aws ec2 describe-instances --instance-ids $IID `
    --query "Reservations[0].Instances[0].PublicIpAddress" --output text
$az = aws ec2 describe-instances --instance-ids $IID `
    --query "Reservations[0].Instances[0].Placement.AvailabilityZone" --output text
```

Inject, then SSH **immediately**:

POWERSHELL

your laptop

```
aws ec2-instance-connect send-ssh-public-key `
    --instance-id $IID `
    --instance-os-user ubuntu `
    --availability-zone $az `
    --ssh-public-key "file://$env:USERPROFILE/.ssh/ec2-key.pub"

ssh -i "$env:USERPROFILE\.ssh\ec2-key" ubuntu@$ip
```

Then make it permanent, so you never repeat the 60-second dance:

BASH

on the instance

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo "ssh-ed25519 AAAA...your-public-key... my-laptop" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

This needs the security group to allow SSH from the EC2 Instance Connect service range. An open `0.0.0.0/0` rule satisfies it — but once you lock port 22 to your own

IP (§05), use the CLI form above rather than the console button.

AMI	DEFAULT SSH USER
Ubuntu	ubuntu
Amazon Linux	ec2-user
Debian	admin

04 SSH shortcut

```
● C:\USERS\<YOU>\.SSH\CONFIG file

Host ec2
  HostName <PUBLIC_IP>
  User ubuntu
  IdentityFile C:\Users\<you>\.ssh\ec2-key
  IdentitiesOnly yes
  ServerAliveInterval 60
  StrictHostKeyChecking accept-new
```

Now just `ssh ec2`.

THE PUBLIC IP CHANGES ON EVERY STOP/START

A plain `reboot` keeps it; a stop/start does not. Update `HostName` each time:

```
● GIT BASH your laptop

NEWIP=$(aws ec2 describe-instances --instance-ids $IID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text | tr -d
'\r')
sed -i "s/^\( *HostName \).*\/\1$NEWIP/" ~/.ssh/config
```

To pin the address permanently, allocate an Elastic IP. Since AWS began charging for all public IPv4 in Feb 2024, an EIP costs the same ~\$3.65/mo as the auto-assigned address while running — but unlike the auto-assigned one, it *keeps billing when the instance is stopped*.

05 Lock down the firewall

A launch-wizard security group opens SSH to `0.0.0.0/0`. Bots find that within minutes. Restrict port 22 to your IP:

POWERSHELL

your laptop

```
$myip = (Invoke-RestMethod https://checkip.amazonaws.com).Trim()

aws ec2 authorize-security-group-ingress --group-id $SG `
  --ip-permissions "IpProtocol=tcp,FromPort=22,ToPort=22,IpRanges=
[{'CidrIp=$myip/32,Description=my-laptop}]"

aws ec2 revoke-security-group-ingress --group-id $SG `
  --protocol tcp --port 22 --cidr 0.0.0.0/0
```

Verify:

POWERSHELL

your laptop

```
aws ec2 describe-security-groups --group-ids $SG `
  --query 'SecurityGroups[0].IpPermissions[0].{Port:FromPort,Allowed:IpRanges[0].CidrIp}'
--output json
```

THE SECURITY GROUP IS THE FIREWALL

Same machine, same network interface: port 80 open to the world, port 22 open only to you. Getting this wrong is how instances get compromised.

Home IPs rotate. When `ssh ec2` starts timing out for no reason, that's the first thing to check — re-run the `authorize` command with your new IP.

06 VS Code Remote-SSH

Files, terminal, Python interpreter and extensions all run **on the server**; only the UI is local.

```
BASH your laptop
code --install-extension ms-vscode-remote.remote-ssh
```

Connect with `Ctrl` + `Shift` + `P` → **Remote-SSH: Connect to Host** → `ec2`. Or from a terminal: `code --remote ssh-remote+ec2 /home/ubuntu/projects`. Look for a green **SSH: ec2** badge bottom-left; `Ctrl` + ``` then opens a terminal *on EC2*.

The most common Windows failure

```
bash: line 1: powershell: command not found
```

```
[Remote - SSH] remote.SSH.remotePlatform = {}
Running script with connection command: ssh -T -D 60235 ec2 powershell
> bash: line 1: powershell: command not found
> Resolver error
```

VS Code doesn't know the remote OS, so on a Windows client it **assumes the remote is Windows** and types `powershell` at your bash prompt. Normally it shows a "Select the platform of the remote host" quick-pick — but launching from the CLI with `--remote` skips that prompt entirely, so it never gets answered.

Fix it permanently in `%APPDATA%\Code\User\settings.json`:

```
SETTINGS.JSON file
{
  "remote.SSH.remotePlatform": { "ec2": "linux" },
  "remote.SSH.connectTimeout": 60,
```

```
"remote.SSH.defaultExtensions": ["ms-python.python", "ms-toolsai.jupyter"]
}
```

Success looks like this — note `sh`, not `powershell`:

```
remote.SSH.remotePlatform = {"ec2": "linux"}
Running script with connection command: ssh -T -D 60312 ec2 sh
Remote server is listening on port 46219
```

WHERE TO DEBUG

Output panel → "**Remote - SSH**" in the dropdown. On disk:

```
%APPDATA%\Code\logs\
```

```
<timestamp>\window<N>\exthost\output_logging_*\*Remote - SSH.log
```


07 Prepare the server

1 GB RAM is tight — **add swap first**, or pip installs get OOM-killed:

BASH

on the instance

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab # survives reboot
free -h
```

BASH

on the instance

```
sudo apt-get update -qq
sudo apt-get install -y python3-pip python3-venv
mkdir -p ~/projects && cd ~/projects
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install jupyterlab pandas
```

The 8 GB default disk is a trap

Real numbers from this build, on a fresh 8 GB volume:

CONSUMER	SIZE
<code>/usr</code> — the OS	1.8 GB
<code>.vscode-server</code> — server 700 MB + extensions 284 MB	~1.0 GB
<code>projects/venv</code> — JupyterLab + pandas	367 MB
<code>/var</code>	600 MB

That reached **95% full**, at which point kernels die mid-execution and pip installs fail halfway.

● BASH

on the instance

```
sudo apt-get clean && sudo apt-get autoremove -y
rm -rf ~/.cache/pip
sudo journalctl --vacuum-size=50M
du -sh ~/.vscode-server/cli/servers/*      # one ~700 MB dir PER VS Code version
df -h /
```

VS Code keeps a full server download for every version it has ever connected with. Delete all but the current commit — it's named in the Remote-SSH log as `Using commit id <hash> for server`.

Growing the volume

● POWERSHELL

your laptop

```
aws ec2 modify-volume --volume-id $VOL --size 20
```

CHECK THE DEVICE NAME — DO NOT ASSUME IT

Nitro-based instances (t3, t4g, m5, c5 and newer) show `nvme0n1`; older Xen families (t2, m4) show `xvda`. Running `growpart` against the wrong device is a bad time. Use whatever `lsblk` actually reports.

● BASH

on the instance

```
lsblk

# if lsblk shows nvme0n1 with partition nvme0n1p1
sudo growpart /dev/nvme0n1 1 && sudo resize2fs /dev/nvme0n1p1

# if lsblk shows xvda with partition xvda1
sudo growpart /dev/xvda 1 && sudo resize2fs /dev/xvda1

df -h /
```

It resizes live, with no downtime and no data loss. **EBS volumes cannot be shrunk** — this is one-directional.

08 Jupyter notebooks in VS Code — the real Colab replacement

● BASH

on the instance

```
source ~/projects/venv/bin/activate
pip install ipykernel
python -m ipykernel install --user --name projects-venv \
    --display-name "Python (projects venv)"
jupyter kernelspec list      # verify it appears
```

Make sure the Python and Jupyter extensions install **on the remote** via `remote.SSH.defaultExtensions` (see §06). Then open any `.ipynb` in the **SSH: ec2** window → **Select Kernel** → **Jupyter Kernel** → **Python (projects venv)**.

The notebook file, the kernel, the interpreter and every variable in memory live on EC2. VS Code is a viewport. No session timeouts, files persist between sessions, and you control the machine.

Proof you're really on EC2

The instance metadata service answers only from inside an instance, so this hangs forever on a laptop:

● PYTHON

kernel on the instance

```
import urllib.request

def imds(path):
    tok = urllib.request.urlopen(urllib.request.Request(
        "http://169.254.169.254/latest/api/token",
        headers={"X-aws-ec2-metadata-token-ttl-seconds": "60"},
        method="PUT")).read()
    return urllib.request.urlopen(urllib.request.Request(
        "http://169.254.169.254/latest/meta-data/" + path,
        headers={"X-aws-ec2-metadata-token": tok})).read().decode()

for f in ["instance-id", "instance-type", "placement/availability-zone"]:
    print(f"{f:32} {imds(f)}")
```

`169.254.169.254` is a link-local address routable only inside EC2. The two-step token exchange is IMDSv2, required by default on new instances.

IF YOU SPECIFICALLY WANT THE COLAB UI

Install `jupyter_http_over_ws`, run JupyterLab on the instance, tunnel it with `ssh -N -L 8888:localhost:8888 ec2`, then use Colab's **Connect to a local runtime** with the `localhost:8888/?token=...` URL. Colab only accepts `localhost`, which is why the tunnel is mandatory. For most work the VS Code route above is simply better.

09 Serve something to the public internet

This is the part your laptop cannot do.

```
● /ETC/SYSTEMD/SYSTEM/DEMO-SITE.SERVICE file

[Unit]
Description=Demo static site on port 80
After=network.target

[Service]
ExecStart=/usr/bin/python3 -m http.server 80 --directory /home/ubuntu/site
Restart=always
User=root

[Install]
WantedBy=multi-user.target
```

```
● BASH on the instance

sudo systemctl daemon-reload
sudo systemctl enable --now demo-site
systemctl is-active demo-site           # -> active
curl -s -o /dev/null -w '%{http_code}\n' http://localhost/      # -> 200
```

Then open `http://<PUBLIC_IP>` on your phone with wifi off. It loads. Close your laptop — it still loads.

- `--now` starts it; `enable` brings it back **after reboot**
- `Restart=always` relaunches it on crash — test with `sudo pkill -f http.server`
- **Type** `http://` **explicitly**. Phone browsers auto-upgrade to HTTPS (nothing listens on 443), and a bare IP often becomes a search query. Both look exactly like "the server is down."

Debugging gotchas that cost real time

Address already in use means it's *working*

```
OSError: [Errno 98] Address already in use
```

Running the command by hand while the service already holds the port produces this. It proves the port is bound. It is not evidence of a fault.

Don't race the socket

Checking `ss` or `curl` in the same command as `systemctl enable --now` reports "nothing listening" — because Python hasn't finished binding yet. Check as a separate step.

Run server commands on the server

A prompt reading `PS C:\Users\you>` is your laptop; `ubuntu@ip-172-31-17-135:~$` is EC2. Pasting `sudo ...` into PowerShell just errors — and a staged unit file that was never moved into `/etc/systemd/system/` means nothing is running at all.

Full sweep when something is unreachable:

● BASH

on the instance

```
systemctl is-active demo-site      # is the service up?
ss -tln | grep ':80'              # is anything bound?
curl http://localhost/             # does it work locally?
sudo ufw status                    # host firewall blocking?
# then: security group rules, and confirm the public IP hasn't changed
```

NOT FOR PRODUCTION

`python -m http.server` is a toy — single-threaded, no HTTPS, no access control. Real deployments use nginx in front of gunicorn/uvicorn, plus certbot for TLS, which needs a domain name pointed at the IP.

10 Cost control and budget alarms

us-east-1 pricing:

ITEM	COST	BILLED WHILE STOPPED?
t3.micro compute	\$0.0104/hr · ~\$7.60/mo	No
Public IPv4 address	\$0.005/hr · ~\$3.65/mo	No — released on stop
EBS root disk, gp3	\$0.08/GB-mo → \$0.64 @ 8 GB, \$1.60 @ 20 GB	Yes — always

Stop the instance when you're done. This one habit keeps it near-free.

POWERSHELL

your laptop

```
aws ec2 stop-instances --instance-ids $IID
```

Set a budget alarm on day one

BUDGET.JSON

file

```
{
  "BudgetName": "monthly-5-usd",
  "BudgetLimit": { "Amount": "5", "Unit": "USD" },
  "TimeUnit": "MONTHLY",
  "BudgetType": "COST"
}
```

NOTIFICATIONS.JSON

file

```
[
  { "Notification": { "NotificationType": "ACTUAL", "ComparisonOperator":
"GREATER_THAN",
    "Threshold": 80, "ThresholdType": "PERCENTAGE" },
    "Subscribers": [ { "SubscriptionType": "EMAIL", "Address": "you@example.com" } ] },
  { "Notification": { "NotificationType": "FORECASTED", "ComparisonOperator":
"GREATER_THAN",
    "Threshold": 100, "ThresholdType": "PERCENTAGE" },
    "Subscribers": [ { "SubscriptionType": "EMAIL", "Address": "you@example.com" } ] }
```



```
"Subscribers": [ { "SubscriptionType": "EMAIL", "Address": "you@example.com" } ] }
```

● POWERSHELL

your laptop

```
aws budgets create-budget --account-id $ACCT `
  --budget file://budget.json `
  --notifications-with-subscribers file://notifications.json

aws budgets describe-budgets --account-id $ACCT --output table
```

FORECASTED ALERTS ARE THE VALUABLE ONES

They warn you on day 3 that you're on track to blow the budget, rather than telling you after the money is gone.

`aws ce get-cost-and-usage` (Cost Explorer) returns `AccessDeniedException: User not enabled for cost explorer access` until it's switched on once from the root account in the Billing console. Budgets works without it.

Check which free tier applies under **Billing** → **Free Tier**. Accounts created before ~July 2025 get 750 hrs/month of t2/t3.micro for 12 months; newer accounts get a credit-based free plan instead.

11 Stop vs terminate, and auditing with CloudTrail

	COMPUTE BILLING	YOUR DATA	PUBLIC IP
stop	ends	survives — EBS volume kept	lost
terminate	ends	destroyed forever	lost

THERE IS NO UNDO

`DeleteOnTermination` defaults to **true** on the root volume. Terminating an instance silently deletes the disk — code, venvs, installed packages, config, everything. No undo, no recycle bin.

Protect a volume you care about:

● POWERSHELL

your laptop

```
aws ec2 modify-instance-attribute --instance-id $IID `
  --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{
  {"DeleteOnTermination":false}}]'
```

Use the device name from `describe-instances` — it may be `/dev/sda1` or `/dev/xvda` depending on the AMI. Or just snapshot before anything risky: `aws ec2 create-snapshot --volume-id $VOL --description "before experiment"`.

Find out who did what

CloudTrail records every API call, searchable for 90 days at no cost:

● POWERSHELL

your laptop

```
aws cloudtrail lookup-events --start-time 2026-08-31T16:00:00Z --max-results 40 `
  --query 'Events[].{Time:EventTime,Event:EventName,User:Username}' --output table
```

The `Username` column is the useful part — `root` means someone was clicking in the console, your IAM username means it came from a script or the CLI. That's how

you establish whether a resource died from a click or a command:

```
23:33:27 StopInstances      dev-user    <- CLI
23:33:46 TerminateInstances root        <- console
```

DON'T OPERATE AS ROOT

AWS recommends the root account only for account-level tasks — billing settings, closing the account. Do daily work as an IAM user with MFA enabled. Root has no permission boundaries and cannot be restricted.

12 The stop routine — how to actually stop paying

Section 11 explains what `stop` means. This is the habit that turns it into money saved. Compute bills per second, so a box you stop after class costs a few cents a day instead of \$7.60 a month.

End of every session

POWERSHELL

your laptop

```
aws ec2 stop-instances --instance-ids $IID
aws ec2 wait instance-stopped --instance-ids $IID
aws ec2 describe-instances --instance-ids $IID `
  --query 'Reservations[].Instances[].State.Name' --output text
```

WAIT FOR STOPPED, NOT STOPPING

`stop-instances` returns immediately with `"CurrentState": "stopping"` — that is the API acknowledging your request, not the machine being off. Shutdown takes 30–60 seconds and you are billed for all of it. `wait instance-stopped` is what turns it into a fact.

You can also stop it from inside the box. For an EBS-backed instance the default shutdown behaviour is `stop`, so `sudo shutdown -h now` bills identically.

What still costs money while stopped

	STOPPED	NOTE
Compute	\$0	this is the whole point
EBS root disk	\$0.64–\$1.60/mo	bills 24/7, stopped or not
Auto-assigned public IP	\$0	released on stop — you get a new one on start
Elastic IP	~\$3.65/mo	keeps billing while the instance is stopped

THE ELASTIC IP CATCHES PEOPLE

Since February 2024 AWS charges for every public IPv4 address, including an Elastic IP sitting idle. If you allocated one for a stable address, a stopped instance still costs ~\$3.65/month for it — more than the disk. Find strays with `aws ec2 describe-addresses` and release what you don't need.

So a stopped dev box costs **about \$1.60/month**, essentially all disk. If that still bothers you, terminate instead and relaunch next time — section 11 covers what you lose.

Starting it back up

● POWERSHELL

your laptop

```
aws ec2 start-instances --instance-ids $IID
aws ec2 wait instance-running --instance-ids $IID
aws ec2 describe-instances --instance-ids $IID `
  --query 'Reservations[].Instances[].PublicIpAddress' --output text
```

THE PUBLIC IP CHANGES ON EVERY START

Your old `~/.ssh/config` entry and any bookmarked URL now point at an address AWS has handed to someone else. Update the `HostName` line from section 4 before reconnecting — an SSH timeout after a restart is almost always this, not a broken instance or firewall.

`wait instance-running` also returns before sshd is ready. A refused connection in the first ~30 seconds is normal; the OS is still booting. Retry before you start debugging.

Never forget again

The reliable fix is not discipline, it's a timer. Set it on the box itself and it shuts down at 22:00 whether or not you remembered:

● BASH

on the instance

```
echo "0 22 * * * root /sbin/shutdown -h now" | sudo tee /etc/cron.d/nightly-stop
```

Sweep for forgotten instances

The expensive mistake is not a box you left running — it's one you left running **in a region you forgot you used**. The console shows one region at a time, so an instance launched in `ap-south-1` during a tutorial is invisible from `us-east-1` forever.

● POWERSHELL

your laptop

```
foreach ($r in (aws ec2 describe-regions --query 'Regions[].RegionName' --output text).Split()) {  
    $out = aws ec2 describe-instances --region $r `  
        --filters "Name=instance-state-name,Values=running" `  
        --query 'Reservations[].Instances[].InstanceId' --output text  
    if ($out) { Write-Output "$r : $out" }  
}
```

Silence means nothing is running anywhere. Run it whenever a bill surprises you, and once at the end of term.

§ Command cheat sheet

● POWERSHELL

your laptop

```
# --- lifecycle ---
aws ec2 start-instances      --instance-ids $IID
aws ec2 stop-instances      --instance-ids $IID
aws ec2 terminate-instances --instance-ids $IID  # PERMANENT, deletes the disk

# wait until genuinely ready (not merely "running")
aws ec2 wait instance-status-ok --instance-ids $IID

# current public IP
aws ec2 describe-instances --instance-ids $IID `
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text

# everything you own, at a glance
aws ec2 describe-instances `
  --query 'Reservations[].Instances[.
{ID:InstanceId,Type:InstanceType,State:State.Name,IP:PublicIpAddress}' `
  --output table

# what else is still billing?
aws ec2 describe-volumes --query 'Volumes[. {ID:VolumeId,GiB:Size,State:State}' --
output table
aws ec2 describe-addresses --query 'Addresses[. {IP:PublicIp,Assoc:InstanceId}' --
output table
aws ec2 describe-snapshots --owner-ids $ACCT --query 'Snapshots[.
{ID:SnapshotId,GiB:VolumeSize}' --output table

# re-allow SSH after your home IP changes
$myip = (Invoke-RestMethod https://checkip.amazonaws.com).Trim()
aws ec2 authorize-security-group-ingress --group-id $SG `
  --protocol tcp --port 22 --cidr "$myip/32"
```

● BASH

on the instance

```
free -h          # memory + swap
df -h /          # disk
lsblk            # block devices (before growpart!)
systemctl is-active <svc>  # is it up?
```


§ Hard-won lessons

- 01 `icacls` needs the full `machine\user` — Git Bash's `whoami` and PowerShell's `$env:USERNAME` both give you only half of it.
 - 02 `export MSYS_NO_PATHCONV=1` before any Git Bash command with a `/leading/path` argument, or it silently becomes `C:/Program Files/Git/...`.
 - 03 `remote.SSH.remotePlatform` must be set, or VS Code tries `powershell` on Linux.
 - 04 `Address already in use` = success, not failure.
 - 05 Never check a socket in the same breath as starting it.
 - 06 Know which machine your prompt is on before pasting commands.
 - 07 Add swap on any 1 GB instance before pip installing anything.
 - 08 Run `lsblk` before `growpart` — device names differ between Nitro and Xen.
 - 09 Never hardcode an AMI id. Resolve it from an SSM parameter; they change constantly.
 - 10 The public IP changes on stop/start (a plain `reboot` keeps it). If SSH times out, check the IP before anything else.
 - 11 `terminate` deletes your disk. `stop` keeps it. Learn this on a box you don't care about.
 - 12 CloudTrail tells you who did what — `root` means the console, your IAM username means a script.
 - 13 Set the budget alarm on day one, not after the first surprise bill.
-

14 **Provision 20 GB, not 8.** The default is smaller than a working dev box needs.